

Étude du Chiffrement Totalement Homomorphe : De DGHV à TFHE et CKKS

Constantin Delahodde, Martin Chedozeau, Lucas Schaeffer

Année Universitaire 2025-2026

Table des matières

1	Introduction	3
1.1	Contexte historique et évolution	3
1.1.1	Une quête de trente ans : des prémices à la rupture de Gentry	3
1.1.2	De la théorie à l'efficacité : vers une cryptographie multi-facettes	3
1.2	Outils théoriques et prérequis	4
1.2.1	Circuits booléens et portes logiques	4
1.2.2	Lien avec le chiffrement homomorphe	5
2	Le schéma DGHV : Chiffrement sur les entiers	6
2.1	Le problème AGCD	6
2.2	Chiffrement Symétrique ("Somewhat Homomorphic")	7
2.3	Chiffrement Asymétrique	7
2.4	Propriétés homomorphes et dynamique du bruit	8
2.5	Le Bootstrapping	9
2.5.1	Le concept de Bootstrapping : « Déchiffrer sans voir »	9
2.5.2	La technique du Squashing : Alléger le déchiffrement	9
3	TFHE : Chiffrement sur le Tore	11
4	CKKS : Calcul sur les nombres réels	12
4.1	Introduction	12
4.2	Fondements mathématiques	12
4.2.1	Anneaux de polynômes	12
4.2.2	Les idées de bruit, d'approximation et d'encodage	13
4.3	L'encodage CKKS	14
4.3.1	Des polynômes pour vectoriser	14
4.3.2	Une écriture simplifiée de l'encodage	14
4.4	Description détaillée de CKKS	14
4.4.1	Aperçu général	14
4.4.2	Génération des clés	15
4.4.3	Encodage et chiffrement	15
4.4.4	Déchiffrement	16
4.4.5	Addition homomorphe	16
4.4.6	Multiplication homomorphe	16
4.5	Bootstrapping	17
4.5.1	Le problème du bruit	17
4.5.2	Principe du bootstrapping	17
4.6	Limites	18
4.6.1	Précision approximative	18

4.6.2	Coût computationnel	18
4.6.3	Gestion du bruit	18
4.7	Conclusion	18

Chapitre 1

Introduction

1.1 Contexte historique et évolution

La confidentialité des données numériques repose traditionnellement sur un compromis binaire : la donnée est soit sécurisée (chiffrée et illisible), soit utile (en clair et traitable). Ce paradigme impose que toute manipulation de données sensibles nécessite une phase de déchiffrement, créant ainsi une faille de sécurité critique au moment même où l'information est traitée en mémoire vive. Le **Chiffrement Totalement Homomorphe** (FHE, pour *Fully Homomorphic Encryption*) a été conçu pour briser cette dualité en permettant l'exécution d'algorithmes arbitraires directement sur des cryptogrammes.

1.1.1 Une quête de trente ans : des prémices à la rupture de Gentry

L'histoire du FHE débute en 1978, un an seulement après l'invention du RSA. Ronald Rivest, Leonard Adleman et Michael Dertouzos introduisent le concept de « *privacy homomorphisms* ». Ils pressentent déjà l'immense potentiel d'un système où une opération sur les chiffrés correspondrait à une opération sur les clairs. Si certains schémas précoces présentaient des propriétés homomorphes partielles — comme le RSA (homomorphe pour la multiplication) ou le cryptosystème de Paillier en 1999 (homomorphe pour l'addition) — la capacité à combiner **simultanément** additions et multiplications est restée le « Saint Graal » de la cryptographie pendant trois décennies.

Le verrou technologique saute en 2009 lorsque Craig Gentry, dans sa thèse de doctorat à l'Université de Stanford, publie le premier schéma totalement homomorphe. Sa percée repose sur une idée révolutionnaire : le *bootstrapping*. Puisque chaque opération homomorphe augmente le bruit inhérent au chiffré jusqu'à rendre le déchiffrement impossible, Gentry propose de déchiffrer le texte chiffré *sous sa forme chiffrée* pour réinitialiser ce bruit. Cette preuve d'existence a transformé un rêve théorique en une réalité mathématique, bien que les premiers temps d'exécution se comptaient alors en jours pour une simple opération.

1.1.2 De la théorie à l'efficience : vers une cryptographie multifacettes

À la suite des travaux de Gentry, la recherche s'est orientée vers la réduction de la complexité algorithmique et la diversification des structures mathématiques sous-jacentes.

On distingue alors plusieurs générations de schémas, passant des réseaux idéaux à des problèmes plus stables comme le *Learning With Errors* (LWE). Ce mémoire se propose d'analyser cette évolution à travers trois protocoles emblématiques qui illustrent la maturité actuelle du domaine :

- **Le schéma DGHV (2010)** : Proposé par van Dijk, Gentry, Halevi et Vaikuntanathan, il simplifie la construction de Gentry en remplaçant les réseaux complexes par l'arithmétique des entiers. L'étude de DGHV est pédagogiquement essentielle pour comprendre la gestion du bruit via le problème du diviseur commun approximatif (AGCD).
- **Le schéma TFHE (2016)** : Représentant de la troisième génération, le *Torus FHE* se distingue par son optimisation radicale du *bootstrapping*. En travaillant sur le tore réel, il permet d'évaluer des portes logiques (NAND, XOR) en quelques millisecondes, ouvrant la voie à l'exécution de circuits booléens complexes et de microprocesseurs chiffrés.
- **Le schéma CKKS (2017)** : Conçu par Cheon, Kim, Kim et Song, ce schéma marque un tournant pragmatique. Contrairement aux schémas exacts, CKKS traite le bruit comme une erreur d'arrondi inhérente au calcul en virgule flottante. Cette approche « approximative » le rend incomparablement plus efficace pour le *Machine Learning* et le traitement de données statistiques à grande échelle.

L'enjeu de ce travail sera de démontrer comment ces constructions mathématiques, bien que radicalement différentes dans leurs approches, convergent vers un objectif commun : rendre le traitement de données privées aussi fluide et universel que le calcul en clair.

1.2 Outils théoriques et prérequis

Pour analyser les mécanismes du chiffrement homomorphe, il est nécessaire de définir le modèle de calcul sous-jacent. Les ordinateurs ne manipulent pas des fonctions mathématiques complexes de manière native, mais évaluent des réseaux de portes logiques traitant des données binaires.

1.2.1 Circuits booléens et portes logiques

En informatique théorique, un algorithme peut être modélisé sous la forme d'un circuit booléen, c'est-à-dire un graphe orienté acyclique composé de portes logiques élémentaires.

Définition 1.1 (Portes logiques sur $\mathbb{F}_2$). Une porte logique est une fonction mathématique opérant sur un ou plusieurs bits (0 ou 1). Les deux opérations fondamentales pour le chiffrement homomorphe correspondent à l'arithmétique dans le corps fini $\mathbb{F}_2$:

- **La porte XOR (OU exclusif)** : Elle correspond strictement à l'addition modulo 2. Elle renvoie 1 si et seulement si un seul des deux bits d'entrée vaut 1.
- **La porte AND (ET)** : Elle correspond strictement à la multiplication modulo 2. Elle renvoie 1 si et seulement si les deux bits d'entrée valent 1.

L'intérêt exclusif pour ces deux portes logiques dans l'étude des schémas de chiffrement repose sur un théorème fondamental de l'informatique théorique.

Proposition 1.1 (Universalité du sous-ensemble $\{\text{AND}, \text{XOR}\}$). *L'ensemble de portes logiques $\{\text{AND}, \text{XOR}\}$ est Turing-complet, sous réserve de disposer de la constante 1 (permettant de simuler l'inversion logique NOT via $x \text{ XOR } 1$). Cela implique que toute fonction calculable, quelle que soit sa complexité, peut être traduite en un circuit combinatoire composé uniquement d'additions et de multiplications modulo 2.*

1.2.2 Lien avec le chiffrement homomorphe

Cette universalité définit le critère de succès d'un schéma *Fully Homomorphic Encryption*. Si un protocole cryptographique parvient à simuler l'addition de deux textes chiffrés (équivalent à une porte XOR sur les clairs) et la multiplication de deux textes chiffrés (équivalent à une porte AND sur les clairs) sans destruction de l'information par le bruit, il devient alors théoriquement capable d'évaluer n'importe quel programme informatique à l'aveugle. L'enjeu des schémas DGHV, TFHE et CKKS réside dans les mécanismes mathématiques déployés pour atteindre cette capacité d'évaluation infinie.

Chapitre 2

Le schéma DGHV : Chiffrement sur les entiers

2.1 Le problème AGCD

Principe de sécurité. La sécurité du schéma DGHV repose sur la difficulté du problème du *Plus Grand Commun Diviseur Approximatif* (AGCD).

Problème 2.1 (AGCD). Soit un entier secret p . Si l'on fournit plusieurs multiples exacts

$$x_i = p \cdot q_i,$$

alors un simple calcul de PGCD permet de retrouver p .

En revanche, si l'on ajoute une petite perturbation (ou *bruit*) r_i :

$$x_i = p \cdot q_i + r_i,$$

alors retrouver p devient extrêmement difficile. C'est précisément ce mécanisme qui permet de masquer la clé secrète dans le schéma DGHV.

Exemple 2.1 (Vulnérabilité sans bruit vs protection). Le bruit n'est pas un défaut, mais un élément structurel indispensable.

Sans bruit. Supposons qu'un attaquant intercepte deux chiffrés :

$$x_1 = 91, \quad x_2 = 143.$$

Il calcule alors $\gcd(91, 143) = 13$, et retrouve immédiatement la clé secrète $p = 13$.

Avec bruit. Introduisons maintenant une perturbation :

$$x_1 = 13 \times 7 + 2 = 93, \quad x_2 = 13 \times 11 + 3 = 146.$$

Dans ce cas, $\gcd(93, 146) = 1$.

L'attaque échoue : le bruit détruit la structure algébrique exploitable et garantit l'indistinguabilité des chiffrés.

2.2 Chiffrement Symétrique ("Somewhat Homomorphic")

Définition 2.1 (Paramètres). Soit λ le paramètre de sécurité.

- p : clé secrète, entier impair d'une taille de λ^2 bits.
- $b \in \{0, 1\}$: le bit de message.
- q : grand entier aléatoire ($\sim \lambda^5$ bits).
- r : petit entier aléatoire représentant le bruit ($\sim \lambda$ bits).

Proposition 2.1 (Chiffrement et Déchiffrement). *Le chiffrement de b produit le texte chiffré c suivant :*

$$c = q \cdot p + 2r + b$$

L'extraction s'effectue avec la clé p par :

$$b = (c \bmod p) \bmod 2$$

Démonstration. Puisque p est beaucoup plus grand que $2r + b$, l'opération $c \bmod p$ supprime le multiple $q \cdot p$ et renvoie exactement $2r + b$. Ensuite, comme $b \in \{0, 1\}$ et que $2r$ est un nombre pair, $(2r + b) \bmod 2 = b$. $\square$

2.3 Chiffrement Asymétrique

Définition 2.2 (Génération des clés). — **Clé secrète (sk)** : L'entier impair p .

- **Clé publique (pk)** : Une suite de $\tau + 1$ entiers $(x_0, x_1, \dots, x_\tau)$. Chaque x_i est un "presque multiple" public de p , généré via la formule $x_i = q_i \cdot p + 2r_i$.

La liste est réorganisée pour que x_0 soit le plus grand entier. De plus, x_0 doit être impair, et $x_0 \bmod p$ doit être un nombre pair.

Proposition 2.2 (Chiffrement). *Pour chiffrer un bit b , on choisit un sous-ensemble aléatoire S d'indices parmi la clé publique et un bruit aléatoire r . Le texte chiffré c est calculé par :*

$$c = \left(b + 2r + \sum_{i \in S} x_i \right) \bmod x_0$$

Démonstration. Par définition du modulo x_0 , il existe $k \in \mathbb{Z}$ tel que :

$$c = b + 2r + \sum_{i \in S} x_i + kx_0$$

En remplaçant chaque x_i par son expression $q_i \cdot p + 2r_i$ (y compris pour x_0), on obtient :

$$c = b + 2r + \sum_{i \in S} (q_i \cdot p + 2r_i) + k(q_0 \cdot p + 2r_0)$$

En séparant les sommes et en regroupant par facteurs communs, on aboutit à :

$$c = p \left(\sum_{i \in S} q_i + kq_0 \right) + 2 \left(r + \sum_{i \in S} r_i + kr_0 \right) + b$$

On retrouve ainsi la structure d'un texte chiffré DGHV standard, ce qui justifie que $b = (c \bmod p) \bmod 2$. $\square$

2.4 Propriétés homomorphes et dynamique du bruit

Proposition 2.3 (Évaluation). Soient $c_1 = pq_1 + 2r_1 + b_1$ et $c_2 = pq_2 + 2r_2 + b_2$ deux textes chiffrés.

- $c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (b_1 + b_2)$
- $c_1 \cdot c_2 = p(q_1q_2p + \dots) + 2(2r_1r_2 + r_1b_2 + r_2b_1) + b_1b_2$

L'addition de textes chiffrés correspond à une porte XOR sur les clairs, et la multiplication à une porte AND.

Définition 2.3 (Budget de bruit et Seuil critique). Chaque schéma de chiffrement homomorphe possède un budget de bruit. Le déchiffrement n'est correct que si la valeur absolue du bruit accumulé respecte le seuil critique :

$$|2r + b| < \frac{p}{2}$$

Si ce seuil est franchi, l'opération modulo p (qui mathématiquement est un modulo centré ramenant les valeurs dans l'intervalle $]-p/2, p/2]$) va altérer la parité du résultat, rendant la récupération du bit impossible.

Exemple 2.2 (Déchiffrement correct vs échec). Soit une clé secrète $p = 101$ (le seuil critique vaut donc $\frac{p}{2} = 50,5$).

- **Déchiffrement correct.** On chiffre un bit $b = 1$ avec un bruit $2r = 10$. Le chiffré vaut $c = 2000 \times 101 + 10 + 1 = 202011$.

On calcule ensuite

$$c \bmod p = 202011 \bmod 101 = 11, \quad \text{d'où } 11 \bmod 2 = 1.$$

Le bit est correctement retrouvé car $11 < 50,5$.

- **Échec du déchiffrement.** On chiffre un bit $b = 0$ et, après plusieurs opérations, le bruit devient $2r = 64$. Le chiffré est alors $c = 2000 \times 101 + 64 + 0 = 202064$.

Lors du déchiffrement, la réduction modulo centré donne

$$[c]_{101} = 64 \equiv -37 \pmod{101} \quad \text{car } 64 > 50,5.$$

Puis

$$-37 \bmod 2 = 1 \neq 0.$$

Le bit est donc corrompu : le déchiffrement échoue.

Proposition 2.4 (Dynamique de la croissance du bruit). L'évaluation homomorphe consomme le budget de bruit, mais de manière très asymétrique :

- **Croissance linéaire (Addition)** : Le bruit du chiffré somme est $r_{add} = r_1 + r_2$. Cette opération est peu coûteuse.
- **Croissance explosive (Multiplication)** : Le produit $c_1 \cdot c_2$ génère un terme de bruit $R = 2(2r_1r_2 + r_1b_2 + r_2b_1) + b_1b_2$. Le nouveau bruit dominant est proportionnel à r_1r_2 , ce qui double approximativement la taille du bruit en bits à chaque multiplication.

Définition 2.4 (Profondeur multiplicative). La profondeur multiplicative désigne le nombre maximal de multiplications successives qu'un circuit peut appliquer à un chiffré avant que son budget de bruit ne dépasse le seuil critique $p/2$. Cette limite distingue le chiffrement *Somewhat Homomorphic* du chiffrement *Fully Homomorphic*.

2.5 Le Bootstrapping

Jusqu'ici, nous avons vu que le schéma DGHV est limité : chaque opération consomme du « budget de bruit ». Une fois ce budget épuisé, le message est perdu. Un tel schéma est dit *Somewhat Homomorphic* (partiellement homomorphe).

Pour devenir *Fully Homomorphic* (totalement homomorphe) et permettre des calculs infinis, il faut pouvoir « nettoyer » le bruit d'un chiffré avant qu'il ne dépasse le seuil critique. C'est le rôle du **bootstrapping**.

2.5.1 Le concept de Bootstrapping : « Déchiffrer sans voir »

L'idée de Craig Gentry (2009) est de rafraîchir un chiffré bruité en lui appliquant la fonction de déchiffrement de manière homomorphe.

Définition 2.5 (Le principe de l'auto-référence). Pour nettoyer un chiffré c devenu trop bruité, le serveur effectue un calcul homomorphe qui simule l'opération de déchiffrement. Pour cela, l'utilisateur doit fournir au serveur une **clé de bootstrapping** : il s'agit de la clé secrète sk elle-même chiffrée ($Enc_{pk}(sk)$).

Le serveur possède alors :

- Un message m enfermé dans un coffre très bruité (c).
- La clé du coffre enfermée dans un autre coffre tout neuf ($Enc(sk)$).

En utilisant les propriétés homomorphes, le serveur utilise la « clé chiffrée » pour ouvrir le « coffre bruité » à l'intérieur d'un nouveau « coffre propre ». Le résultat est un nouveau chiffré du même message m , mais avec un bruit réinitialisé.

2.5.2 La technique du Squashing : Alléger le déchiffrement

Le principal obstacle au *bootstrapping* dans le schéma DGHV est la complexité de la fonction de déchiffrement originale. En effet, pour retrouver le message, il faut calculer :

$$m = (c \pmod{p}) \pmod{2}$$

Ce calcul nécessite une division euclidienne par la clé secrète p , une opération dont la complexité multiplicative est trop élevée pour le budget de bruit du schéma. Pour résoudre ce problème, Craig Gentry a introduit le **squashing** (écrasement du circuit), qui consiste à simplifier la procédure de déchiffrement.

Principe : Remplacer la division par une somme

L'idée du *squashing* est de pré-calculer une partie de la division "difficile" et de la fournir au serveur sous forme d'indices cachés. Le processus se décompose en trois étapes :

1. **La clé publique augmentée** : On génère un grand ensemble $\vec{y} = \{y_1, y_2, \dots, y_n\}$ de nombres rationnels compris entre 0 et 2. Cet ensemble est public.
2. **La clé secrète simplifiée (Vecteur creux)** : L'utilisateur définit un vecteur secret $\vec{s} = \{s_1, s_2, \dots, s_n\}$ composé de bits (0 ou 1). Ce vecteur est dit **très creux** (*sparse*) car il ne contient qu'un très petit nombre d'éléments égaux à 1 (noté κ , où $\kappa \ll n$).

3. **Le secret partagé :** Ces éléments sont choisis de telle sorte que leur somme soit une approximation de l'inverse de la clé secrète :

$$\sum_{i=1}^n s_i y_i \approx \frac{1}{p} \pmod{2}$$

Nouveau circuit de déchiffrement

Grâce à cette préparation, le serveur ne calcule plus une division, mais une somme pondérée. Pour déchiffrer, le serveur calcule d'abord en clair les valeurs intermédiaires $z_i = (c \cdot y_i) \pmod{2}$. Le déchiffrement devient alors :

$$b = \left(c - \left\lfloor \sum_{i=1}^n s_i z_i \right\rfloor \right) \pmod{2}$$

Pourquoi cette méthode est-elle efficace ?

L'avantage majeur du *squashing* réside dans la réduction du bruit généré lors de l'opération :

- **Faible complexité :** Puisque le vecteur $\vec{s}$ est très creux, la somme $\sum s_i z_i$ ne comporte en réalité que κ termes (par exemple, 15 additions au lieu de 1000).
- **Compatibilité avec le bootstrapping :** Cette somme est suffisamment "légère" pour que le bruit créé par son calcul homomorphe reste inférieur au seuil critique $p/2$.

Chapitre 3

TFHE : Chiffrement sur le Tore

Chapitre 4

CKKS : Calcul sur les nombres réels

Le schéma CKKS (*Cheon–Kim–Kim–Song*) est un chiffrement homomorphe *approximatif* conçu pour effectuer des calculs sur des nombres réels ou complexes chiffrés. Contrairement aux schémas exacts, il accepte une petite erreur de représentation et d'arrondi, ce qui le rend plus adapté aux applications où une précision flottante suffit, par exemple en apprentissage automatique ou en calcul scientifique privé. Ce chapitre présente les idées mathématiques de base ainsi que le fonctionnement du schéma.

4.1 Introduction

Les premiers schémas homomorphes ne permettaient que des opérations très limitées. CKKS, introduit par Cheon, Kim, Kim et Song [1], est particulièrement intéressant car il travaille avec des nombres *approximatifs* au lieu d'exiger une exactitude parfaite.

L'intérêt principal de CKKS est qu'il permet de calculer sur des valeurs réelles ou complexes encodées sous forme de polynômes. En pratique, cela convient très bien à des domaines où l'on manipule déjà des données approchées : statistiques, filtrage, apprentissage automatique, calcul numérique, etc. Il y a cependant une petite erreur numérique toujours présente qui augmente au fil des opérations.

4.2 Fondements mathématiques

4.2.1 Anneaux de polynômes

Un polynôme est une expression de la forme

$$p(X) = a_0 + a_1X + \cdots + a_dX^d,$$

où les coefficients a_i appartiennent à un ensemble donné, par exemple $\mathbb{Z}$ ou $\mathbb{Z}_q$.

Dans CKKS, on utilise un anneau du type

$$R_q = \mathbb{Z}_q[X]/(X^N + 1),$$

où :

- N est généralement une puissance de 2 ;
- $\mathbb{Z}_q$ désigne les entiers modulo q ;

- l'écriture modulo $(X^N + 1)$ signifie que deux polynômes sont considérés comme égaux dès que leur différence est multiple de $X^N + 1$.

Dans cet anneau, on fait les calculs comme d'habitude sur les polynômes, puis on réduit le résultat modulo $X^N + 1$ et modulo q .

Exemple 4.1 (Multiplication dans un anneau quotient). Prenons

$$R_{17} = \mathbb{Z}_{17}[X]/(X^2 + 1).$$

Calculons

$$(3 + 5X)(7 + 4X).$$

On développe :

$$(3 + 5X)(7 + 4X) = 21 + 12X + 35X + 20X^2 = 21 + 47X + 20X^2.$$

Dans l'anneau, on a $X^2 \equiv -1$, donc

$$20X^2 \equiv -20 \equiv 14 \pmod{17}.$$

Ainsi

$$21 + 47X + 20X^2 \equiv 21 + 47X + 14 \equiv 1 + 13X \pmod{17}.$$

On obtient donc

$$(3 + 5X)(7 + 4X) \equiv 1 + 13X \pmod{X^2 + 1, 17}.$$

Cet exemple montre l'idée essentielle : le calcul se fait d'abord comme sur les polynômes ordinaires, puis on ramène le résultat dans une forme standard grâce aux réductions modulo.

4.2.2 Les idées de bruit, d'approximation et d'encodage

Dans CKKS, un message chiffré n'est pas stocké de manière exacte. Il est représenté avec un petit *bruit* numérique ou algébrique, indispensable pour la sécurité.

Définition 4.1 (Encodage et Décodage). On appelle *encodage* la transformation qui convertit un message numérique classique, généralement un vecteur de nombres réels ou complexes, en une représentation adaptée au schéma de chiffrement utilisé (CKKS).

Inversement, le *décodage* est l'opération qui, à partir d'une donnée encodée, permet de retrouver une approximation du message original.

Définition 4.2 (Bruit). On appelle *bruit* la petite erreur présente dans un chiffré et qui fait que le déchiffrement ne redonne pas directement la valeur encodée, mais une valeur très proche.

Dans CKKS, on accepte ce bruit et on l'interprète comme une erreur d'approximation. L'idée est alors la suivante :

$$\text{valeur exacte} \longrightarrow \text{valeur encodée} \longrightarrow \text{chiffré}$$

puis, après calcul,

$$\text{chiffré calculé} \longrightarrow \text{valeur décodée approximative}.$$

Pour contrôler cette approximation, on introduit un *facteur d'échelle* Δ , souvent une grande puissance de 2. Les nombres réels sont multipliés par Δ avant d'être arrondis à des entiers. À la fin, on divise par Δ pour revenir à l'échelle initiale.

4.3 L'encodage CKKS

4.3.1 Des polynômes pour vectoriser

CKKS ne chiffre pas un seul nombre à la fois : il peut *conditionner* plusieurs valeurs dans un seul chiffré. Dans la version classique, on peut stocker environ $N/2$ nombres complexes dans un chiffré basé sur l'anneau R_q .

L'anneau des polynômes n'est pas vu seulement comme des coefficients, mais aussi comme des valeurs du polynôme en certaines racines de l'unité. On passe d'une représentation *par coefficients* à une représentation *par valeurs*.

4.3.2 Une écriture simplifiée de l'encodage

Définition 4.3 (Encodage dans CKKS). CKKS encode un vecteur $\mathbf{z} = (z_1, \dots, z_{n/2}) \in \mathbb{C}^{n/2}$ en un polynôme $m(X) \in R$ tel que les évaluations de m aux racines primitives $2n$ -ièmes de l'unité approchent les composantes de $\mathbf{z}$.

Si l'on note σ l'application d'encodage qui transforme un vecteur de nombres complexes en un polynôme qui approxime les valeurs de $\mathbf{z}$ aux racines primitives $2n$ -ièmes de l'unité, alors l'encodage s'écrit schématiquement :

$$\text{Encodage}_\Delta(\mathbf{z}) = \lfloor \Delta \cdot \sigma(\mathbf{z}) \rfloor \in R,$$

où :

- $\mathbf{z}$ est un vecteur de nombres à chiffrer ;
- Δ est l'échelle ;
- $\lfloor \cdot \rfloor$ désigne l'arrondi coefficient par coefficient ;
- $R = \mathbb{Z}[X]/(X^N + 1)$ est la version entière de l'anneau.

Concrètement, σ est une interpolation polynomiale (*proche d'une transformée de Fourier discrète*).

Le point important est le suivant : *on encode avec un arrondi, donc on accepte une petite erreur dès le départ*. CKKS est donc, dès l'encodage, un schéma approximatif.

4.4 Description détaillée de CKKS

4.4.1 Aperçu général

CKKS peut être vu comme un schéma à quatre grandes étapes :

1. génération des clés ;
2. encodage et chiffrement ;
3. calcul homomorphe ;
4. déchiffrement et décodage.

Algorithm 1 Version simplifiée de CKKS

- 1: **Entrée** : un vecteur de nombres $\mathbf{z}$, une échelle Δ
 - 2: Générer une paire de clés (pk, sk)
 - 3: Calculer $m \leftarrow \text{Encodage}_\Delta(\mathbf{z})$
 - 4: Chiffrer m pour obtenir $\mathbf{c}$ le message chiffré
 - 5: Effectuer des additions et multiplications homomorphes sur $\mathbf{c}$
 - 6: Si nécessaire, appliquer relinéarisation, rescaling et bootstrapping
 - 7: Récupérer le résultat : $m^* \leftarrow \text{Dec}(\mathbf{c})$
 - 8: Retourner $\text{Decode}(m^*/\Delta)$
-

4.4.2 Génération des clés

On travaille dans un anneau $R_q = \mathbb{Z}_q[X]/(X^N + 1)$.

1. On choisit une clé secrète s “petite”, c’est-à-dire un polynôme de faible norme, souvent à coefficients dans $\{-1, 0, 1\}$.
2. On choisit un polynôme aléatoire a dans R_q .
3. On choisit un petit bruit e .
4. On définit la clé publique par

$$\text{pk} = (a, b), \quad b = -as + e \pmod{q}.$$

L’idée est que la paire (a, b) cache s à cause du bruit e . La sécurité repose sur la difficulté du problème RLWE (*Ring Learning With Errors*).

Remarque. Les coefficients des polynômes “petits” $(e, s, \dots)$ suivent une loi normale.

4.4.3 Encodage et chiffrement

Soit un message réel ou complexe $\mathbf{z}$. On commence par l’encoder :

$$m = \text{Encodage}_\Delta(\mathbf{z}).$$

Le résultat m est un polynôme à coefficients entiers, mais il représente approximativement les valeurs de $\mathbf{z}$.

Pour chiffrer m :

1. on choisit un polynôme aléatoire u très petit ;
2. on choisit deux bruits petits e_1 et e_2 ;
3. on calcule

$$c_0 = bu + e_1 + m \pmod{q}, \quad c_1 = au + e_2 \pmod{q}.$$

Le chiffré est alors le couple

$$\mathbf{c} = (c_0, c_1) \in R_q^2.$$

4.4.4 Déchiffrement

En remplaçant b par $-as + e$, on obtient :

$$c_0 + c_1s = (bu + e_1 + m) + (au + e_2)s = eu - asu + e_1 + m + asu + e_2s = m + (eu + e_1 + e_2s).$$

Le terme entre parenthèses est petit. Donc, après déchiffrement, on retrouve m à une petite erreur près.

Il suffit ensuite de décoder ce polynôme pour retrouver un vecteur de nombres approchés.

4.4.5 Addition homomorphe

Si

$$\mathbf{c} = (c_0, c_1) \quad \text{et} \quad \mathbf{d} = (d_0, d_1)$$

sont deux chiffrés, alors leur somme est définie coefficient par coefficient :

$$\mathbf{c} + \mathbf{d} = (c_0 + d_0, c_1 + d_1).$$

En effet, lors du déchiffrement :

$$(c_0 + d_0) + (c_1 + d_1) \cdot s = (c_0 + c_1s) + (d_0 + d_1s) \approx m + m'$$

Un point important est que l'addition n'augmente *presque pas* le bruit du chiffrement. Elle effectue la somme des 2 bruits gardant l'erreur assez faible.

4.4.6 Multiplication homomorphe

La multiplication est plus délicate. On définit la multiplication de deux chiffrés de taille 2 (*multiplication de polynômes*) par :

$$(c_0, c_1) \cdot (d_0, d_1) = (c_0d_0, c_0d_1 + c_1d_0, c_1d_1) := (x_0, x_1, x_2)$$

Pour déchiffrer, on effectue :

$$x_0 + x_1s + x_2s^2 = c_0d_0 + (c_0d_1 + c_1d_0)s + c_1d_1s^2 = (c_0 + c_1s)(d_0 + d_1s) \approx m \cdot m'.$$

Ce qui correspond bien au produit des messages. Mais le problème est que le degré du chiffré augmente (*ici de 2 à 3*).

Relinéarisation. Après une multiplication homomorphe, on obtient un chiffré de la forme (c_0, c_1, c_2) correspondant à :

$$m \approx c_0 + c_1s + c_2s^2$$

Le terme en s^2 empêche un déchiffrement direct classique.

Pour résoudre ce problème, on utilise une *clé de relinéarisation*, notée (rk_0, rk_1) , qui est un chiffrement de s^2 :

$$rk_0 + rk_1s \approx s^2$$

On remplace alors :

$$c_2s^2 \approx c_2(rk_0 + rk_1s)$$

Ainsi :

$$m \approx (c_0 + c_2 r k_0) + (c_1 + c_2 r k_1) s$$

On définit donc un nouveau chiffré :

$$(c'_0, c'_1) = (c_0 + c_2 r k_0, c_1 + c_2 r k_1)$$

Vérifiant :

$$c'_0 + c'_1 \cdot s \approx c_0 + c_1 s + c_2 s^2 \approx m$$

Ce chiffré est de nouveau linéaire en s , ce qui permet de continuer les calculs homomorphes.

Rescaling. Dans CKKS, chaque message m est encodé avec une échelle Δ pour convertir des nombres réels en entiers modulaires :

$$\tilde{m} = \lfloor m \cdot \Delta \rfloor$$

Après une multiplication homomorphe :

$$\tilde{m}_1 \cdot \tilde{m}_2 \approx m_1 m_2 \cdot \Delta^2$$

L'échelle a donc doublé, ce qui peut provoquer saturation et perte de précision.

Le *rescaling* consiste à diviser le chiffré par Δ pour revenir à une échelle contrôlable :

$$c_{\text{rescaled}} = \left\lfloor \frac{c}{\Delta} \right\rfloor \approx m_1 m_2 \cdot \Delta$$

Le rescaling permet de garder les valeurs numériques sous contrôle tout en conservant la précision relative.

4.5 Bootstrapping

4.5.1 Le problème du bruit

Chaque opération homomorphe augmente le bruit. À terme, si le bruit devient trop grand, le déchiffrement échoue : le terme utile est dépassé par les erreurs.

On ne peut donc effectuer qu'un nombre fini d'opérations avant de perdre la précision ou la validité du déchiffrement.

4.5.2 Principe du bootstrapping

Le *bootstrapping* est une méthode de “rafraîchissement” des chiffrés. L'idée, héritée de Gentry, est de faire calculer par le système homomorphe une version approximative de sa propre fonction de déchiffrement, afin de produire un nouveau chiffré contenant moins de bruit.

Dans CKKS, cette étape est particulièrement subtile, car il faut :

- gérer le bruit déjà présent ;
- maintenir une bonne précision numérique ;
- réajuster l'échelle ;
- parfois utiliser des transformations de type FFT entre espace des coefficients et espace des slots.

Le bootstrapping permet donc d'enchaîner davantage d'opérations qu'un schéma purement limité, mais il reste coûteux.

4.6 Limites

4.6.1 Précision approximative

CKKS ne donne pas un résultat exact. À la place, il fournit une approximation contrôlée. Cela convient à beaucoup d'applications, mais pas à toutes. Un problème exact n'est pas le bon cadre pour CKKS.

4.6.2 Coût computationnel

Par rapport à un calcul non chiffré, CKKS est très coûteux :

- les polynômes sont de très grand degré ;
- les modules ont des tailles importantes ;
- les multiplications demandent des opérations de réécriture, de relinéarisation et de rescaling ;
- le bootstrapping est encore plus lourd.

Cependant, CKKS permet des calculs plus rapides que des méthodes de calculs homomorphes exactes (DGHV/TFHE) notamment grâce à ses calculs vectorisés.

4.6.3 Gestion du bruit

La gestion du bruit est le cœur pratique du schéma. Il faut choisir :

- la taille de l'échelle Δ ;
- le nombre de niveaux autorisés ;
- la taille du module q ;
- les paramètres de sécurité ;
- la stratégie de bootstrapping si elle est nécessaire.

Il existe donc un compromis permanent entre sécurité, précision et vitesse.

4.7 Conclusion

CKKS est aujourd'hui l'un des schémas homomorphes approximatifs les plus importants. Son originalité est d'accepter volontairement une petite erreur numérique afin de permettre des calculs efficaces sur des nombres réels ou complexes. Cette idée est très pertinente en pratique, notamment pour les calculs statistiques et l'apprentissage automatique privé.

Du point de vue mathématique, CKKS repose sur l'arithmétique modulaire ainsi que les anneaux de polynômes. Du point de vue algorithmique, il introduit des opérations homomorphes bien structurées, mais qui nécessitent une gestion de l'échelle et du bruit.

Bibliographie

- [1] Jung Hee CHEON et al. « Homomorphic Encryption for Arithmetic of Approximate Numbers ». In : *Advances in Cryptology – ASIACRYPT 2017*. Springer, 2017, p. 409-437.
- [2] Ilaria CHILLOTTI et al. « TFHE : Fast Fully Homomorphic Encryption Over the Torus ». In : *Journal of Cryptology* 33 (2020), p. 34-91.
- [3] Marten van DIJK et al. « Fully Homomorphic Encryption over the Integers ». In : *Advances in Cryptology – EUROCRYPT 2010*. Springer, 2010, p. 24-43.
- [4] FHE.ORG COMMUNITY. *History of FHE : A Timeline*. URL : <https://fhe.org/>.
- [5] Craig GENTRY. « A fully homomorphic encryption scheme ». Thèse de doct. Stanford University, 2009.
- [6] Daniel HUYNH. *CKKS Explained : Part 1, Simple Encoding and Decoding*. 2020. URL : <https://openmined.org/blog/ckks-explained-part-1-simple-encoding-and-decoding/>.
- [7] Pascal PAILLIER. *Introduction to Homomorphic Encryption*. FHE.org Meetup Presentation. 2020.
- [8] João Vítor Oliveira PALIARES et João MARTINELLI. *Explaining the DGHV Encryption Scheme*. 2024. URL : <https://medium.com/@j248360/explaining-the-dghv-encryption-scheme-1acb6cd74dd6>.
- [9] Linus TANG. *CKKS Homomorphic Encryption Part 1 — The Original Scheme*. 2024. URL : https://www.mit.edu/~linust/files/CKKS_Homomorphic_Encryption_Part_1.pdf.
- [10] David J. WU. *Fully Homomorphic Encryption : Cryptography’s Holy Grail*. Rapp. tech. UT Austin Computer Science, 2015.